Trust, but Verify: Black-Box Auditing of Opt-Out Compliance in Graph Feature Pipelines

Anonymous Authors

Submitted to the Workshop on Privacy-Preserving Artificial Intelligence (PPAI)

Abstract

Platforms that let users opt out of behavioral data use must ensure that no interaction of an opted-out user influences downstream features. In recommender and marketplace systems, those features are often node embeddings and aggregates derived from a customer–item–merchant interaction graph. We study how an auditor can verify this property when the feature pipeline is a black box, and when the only evidence is a daily sequence of triples $(E^t_{\text{in}}, E^t_{\text{out}}, F^t)$: opted-in edges, opted-out edges and produced node features. We cast compliance as the conditional-independence hypothesis $F \perp\!\!\!\perp E_{\text{out}} \mid E_{\text{in}}$, and we note that no black-box procedure can certify zero use: it can only detect leakage above a threshold and bound what remains. We identify the confounders that make naive tests unsound, including the correlation between opted-in and opted-out behavior, pipeline stochasticity, rotational non-identifiability of embeddings, and the dependence and drift of real behavior. We measure each failure on compliant pipelines. We then propose six passive tests that exploit the problem's structure: natural canaries, zero-difference tests on invariant neighborhoods, opt-out status-switch difference-in-differences, residual regression with surrogate controls, and two membership-inference tests, one on item pairs with a temporal placebo and one on edges with a same-customer reference. An active canary test with randomized assignment serves as a reference. Evidence is combined across days with anytime-valid e-values. In a synthetic study, the calibrated tests keep per-day false-positive rates at or below 4% and raise no false alarm in 30 ten-day audits of a clean pipeline. They detect an aggregation bug that leaks 2% of opted-out edges on day one. Natural canaries make the passive audit as powerful as an active campaign with 80 canary edges a day. On real e-commerce interactions (RetailRocket), tests built on structural facts stay calibrated and detect every simulated leak within two days. Tests built on behavioral exchangeability alarm on 13–29% of compliant days until they use customer-level units and time-bracketed references. Naive tests raise false alarms on up to 100% of compliant days.

1 Introduction

User-level opt-out is a standard privacy control. Regulations such as the GDPR right to object and the CCPA right to opt out make it a legal obligation as well as a design choice. Large platforms often store behavior as a heterogeneous graph (Zhu et al., 2019; Sarda et al., 2019; Borisyuk et al., 2024). Customers connect to items through views, saves, purchases and reviews, and items connect to the merchants that sell them. Downstream models seldom consume this graph directly. They consume *node-level features*, such as graph embeddings, GNN outputs (Hamilton, Ying, and Leskovec, 2017; Ying et al., 2018; El-Kishky et al., 2022) and hand-crafted aggregates like a restaurant's average preparation time (Hermann and Del Balso, 2017). A feature pipeline produces these features, typically once a day. Section 2 documents this practice.

Honoring an opt-out means that no edge incident to an opted-out customer may influence *any* produced feature. That covers the customer's own feature and the features of every item and merchant reachable through their edges. Filtering bugs break this promise silently: a join that drops the consent predicate, a stale cache, or a training job that reads an unfiltered snapshot.

Why not inspect the pipeline? In principle, a privacy team could review the code or rely on data-lineage metadata. In practice, both fall short. *Velocity.* Gating every pipeline change on manual privacy review slows development. Organizations increasingly let teams ship and verify compliance continuously afterwards. *Least privilege.* Giving one team read access to every pipeline and its data concentrates authority, which creates an insider risk and a security target. An auditor that sees only edge lists, consent labels and outputs needs far less access. *Scale and ownership.* A large platform runs hundreds of feature pipelines in SQL, Spark and notebook code. Different teams own them and change them weekly. Some stages are vendor services or managed feature stores whose source is unavailable. External assessors face the same opacity for intellectual-property reasons. *Code and lineage are not behavior.* Compliance depends on runtime state: which version of the consent table was joined, how fresh a cache was, whether a backfill ran, and which snapshot

a training job read. Lineage systems usually track dependencies between tables or columns, not which rows passed a consent predicate. They also miss copies made outside instrumented tools. A code review, finally, judges code at one point in time, and that code keeps changing.

We therefore verify compliance from observable behavior alone, independently of how the pipeline is implemented. This mirrors the separation of duties in financial auditing. The auditor observes only what goes in and what comes out.

We study that setting. Each day the auditor receives $(E^t_{\text{in}}, E^t_{\text{out}}, F^t)$ from a fixed but opaque pipeline and must decide whether F^t was computed from E^t_{in} alone. Our contributions are:

1. A formulation of opt-out compliance as conditional independence, with an explicit limit on what black-box evidence can establish (Section 3).
2. A catalogue of confounders that invalidate naive tests (Section 4).
3. Six complementary passive audit tests that target aggregation leakage and representation leakage, and an active reference test (Section 5).
4. A sequential framework that combines daily evidence with e-values and reports calibrated leakage bounds instead of an unfalsifiable claim of compliance (Section 6).
5. Experiments on synthetic data and on real e-commerce interactions (RetailRocket). They measure how each naive test fails, and they show which tests carry over to real behavior and which need refinements (Section 7).

The framing follows empirical auditing of differential privacy (Jagielski, Ullman, and Oprea, 2020; Nasr et al., 2021; Steinke, Nasr, and Jagielski, 2023). In that setting, black-box experiments yield lower bounds on privacy loss, never proofs of privacy. Here they yield detection guarantees above a threshold, never proofs of compliance.

2 The Setting in Practice

Both halves of our setting describe standard industrial practice. The first half is a typed customer–item–merchant interaction graph. The second is a pipeline that turns that graph into node-level features consumed by other models. Table 1 summarizes published systems.

The graph. Public e-commerce logs record exactly our schema. The Tmall repeat-buyer dataset logs each interaction with a user, item, category, merchant and brand, plus one of four action types: click, add-to-cart, purchase or add-to-favorite (Tianchi, 2015). The Taobao UserBehavior dataset uses page views, carts, favorites and purchases (Tianchi, 2018). PyTorch Geometric (Fey and Lenssen, 2019) ships it as a heterogeneous graph with typed user–item edges. Industrial graphs are larger, with the same shape. AliGraph describes a buyer–seller–commodity graph with hundreds of billions of nodes and edges (Zhu et al., 2019). Uber Eats builds user–dish and user–restaurant bipartite graphs whose edges are weighted by order counts and ratings (Sarda et al., 2019). LinkedIn's graph contains tens of node and edge types (Borisyuk et al., 2024).

Node-level features consumed downstream. Two families dominate. The first is *learned embeddings*. PinSage computes embeddings on a graph of 3 billion nodes and 18 billion edges (Ying et al., 2018). Taobao embeds items together with shop and brand side information (Wang et al., 2018). Uber Eats computes a vector for each user, restaurant and dish (Sarda et al., 2019). TwHIN embeddings serve as frozen features in ads ranking, follow recommendation, search ranking and content classification (El-Kishky et al., 2022). AliGraph states that the generated embeddings are fed as features into downstream tasks (Zhu et al., 2019). Airbnb converts listing embeddings into similarity features for its search ranker (Grbovic and Cheng, 2018). The second family is *aggregates*. Uber's Michelangelo computes features such as a restaurant's average meal preparation time over the last seven days. It publishes them to a feature store that is shared across teams and recomputed daily (Hermann and Del Balso, 2017).

Implications for auditing. Three consequences follow. First, one embedding table or aggregate feeds many downstream models, so a single filtering bug spreads widely. The feature boundary is therefore the natural place to audit, not each downstream model. Second, daily batch recomputation produces exactly the daily triples $(E^t_{\text{in}}, E^t_{\text{out}}, F^t)$ we assume. Third, real systems deviate from our model in three ways, and none of them breaks it:

- Some systems treat the merchant as side information rather than a node (Wang et al., 2018). Merchant features are then functions of item features, and the audit applies unchanged.
- Some learn from sessions rather than explicit edges (Grbovic and Cheng, 2018; Wang et al., 2018). Sessions are built from the same interactions, so the sessions of opted-out customers belong to E_{out}.
- Near-real-time features (Hermann and Del Balso, 2017) call for triples at a finer time granularity. We leave this extension to future work.

3 Problem Formulation

Setup. On day t, let G^t be a typed bipartite graph between customers $\mathcal{U}$ and items $\mathcal{I}$, plus a static item–merchant map $m : \mathcal{I} \to \mathcal{M}$ and intrinsic item embeddings $x_i \in \mathbb{R}^d$. Each customer u has an opt-out status $o^t_u \in \{0, 1\}$. The edges split into $E^t_{\text{in}} = \{(u, i, \tau) : o^t_u = 0\}$ and $E^t_{\text{out}} = \{(u, i, \tau) : o^t_u = 1\}$, where τ is the interaction type. A fixed pipeline f maps

System	Entities in the graph	Interaction edges	Node-level output	Downstream consumer
Uber Eats (Sarda et al., 2019)	users, dishes, restaurants	orders (weighted by count), ratings	GraphSAGE embeddings	candidate generation, personalized ranker
Taobao EGES (Wang et al., 2018)	items (+ shop, brand, category)	co-occurrence in behavior sessions	item embeddings	matching stage of recommendation
AliGraph (Zhu et al., 2019)	buyers, sellers, commodities	user behaviors	GNN embeddings	features for recommendation and search
Pinterest PinSage (Ying et al., 2018)	pins, boards	saves to boards	GCN embeddings	recommendation
Twitter TwHIN (El-Kishky et al., 2022)	users, tweets, advertisers	fave, reply, retweet, follow	HIN embeddings	frozen features in ads, follow, search, safety models
LinkedIn LiGNN (Borisyuk et al., 2024)	members, posts, jobs, companies	engagements, affinities	GNN embeddings	feed, jobs, ads, people recommendation
Airbnb (Grbovic and Cheng, 2018)	users, listings	clicks, bookings (sessions)	listing embeddings	similarity features in search ranking
DoorDash (Koch, 2018)	consumers, stores	orders	store embeddings	store-feed ranking
Uber Michelangelo (Hermann and Del Balso, 2017)	restaurants	orders	aggregates (e.g., 7-day mean prep time)	shared feature store, updated daily
Tmall IJCAI-15 (Tianchi, 2015)	users, items, merchants, brands	click, add-to-cart, purchase, favorite	—	public benchmark
Taobao UserBehavior (Tianchi, 2018)	users, items, categories	pv, cart, fav, buy	—	public benchmark

Table 1: Published systems and datasets that match our setting: typed interaction graphs between customers, items and merchants, turned into node-level embeddings or aggregates that other models consume.

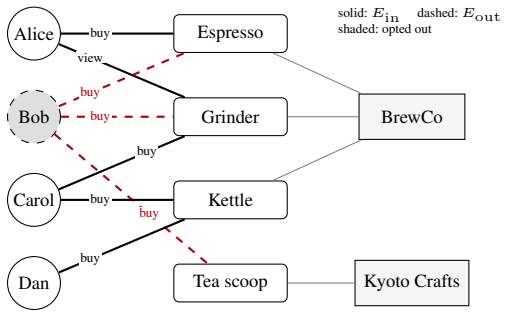


Figure 1: Running example on Monday. Bob has opted out, so his three purchases form E_{out}. The Tea scoop, and with it Kyoto Crafts, was bought only by Bob. Dan will opt out on Tuesday.

its input to node features $F^t = \{F_v^t\}_{v \in \mathcal{U} \cup \mathcal{I} \cup \mathcal{M}}$. The auditor observes $\mathcal{D}_T = \{(E_{\text{in}}^t, E_{\text{out}}^t, F^t)\}_{t=1}^T$ but neither f nor its internal randomness ξ^t.

Running example. Figure 1 shows a toy marketplace that we reuse throughout the paper. Alice, Carol and Dan are opted in, and Bob is opted out. A compliant pipeline must produce exactly the features it would produce if Bob's three dashed edges did not exist. For example, the Grinder's purchase count must be 1 (Carol), not 2. The Tea scoop must look like an item nobody has bought. Kyoto Crafts, whose only sale came from Bob, must look like a merchant with no sales.

Hypothesis. A compliant pipeline satisfies, for some measurable f_0,

$$H_0 : \; F^t = f_0(E_{\text{in}}^t, x, \xi^t) \quad \forall t, \tag{1}$$

with ξ^t independent of E_{out}^t given E_{in}^t. Equivalently, $F^t \perp\!\!\!\perp E_{\text{out}}^t \mid E_{\text{in}}^t, x$. The alternative is that F^t depends on E_{out}^t through some channel. Note that this is a *conditional* statement: F and E_{out} are marginally dependent even under H_0, because opted-in and opted-out customers draw from the same preference distribution.

Leakage modes. We distinguish two channels: *aggregation leakage*, where counts, pooling or message passing include some opted-out edges; and *representation leakage*, where trained parameters (embedding tables, GNN weights, factorization) are fit on data containing opted-out edges, even if inference later uses only E_{in}. The second channel is the harder one to detect. It need not change any feature near an opted-out edge, so tests confined to local neighborhoods can miss it.

What cannot be shown. Could an auditor ever *certify* that a pipeline is compliant? The answer is no, and the reason

shapes everything that follows.

Testing H_0 is a conditional-independence (CI) test: we ask whether F still depends on E_{out} once E_{in} is known. Shah and Peters (2020) prove that CI testing is hard in a strong sense. When the conditioning variable is continuous, any test that keeps its false-positive rate at most α under *every* null distribution has power at most α against *every* alternative. Put simply, a test cannot be both always valid and ever useful unless we add assumptions about what the data can look like. Their result concerns general distributions, and our setting has extra structure (a graph and a fixed pipeline), so we do not rely on it directly. A more elementary argument fits our setting and gives a concrete number.

Picture a buggy pipeline f_1 that behaves exactly like a compliant f_0 except on rare inputs. In Figure 1, f_1 might include Bob's purchases only when a nightly job retries after a timeout and skips the consent join. On every other day the two pipelines produce identical outputs, so no statistic of the outputs can tell them apart on those days.

Proposition 1 (No certificate of zero use)**.** *Let f_0 satisfy H_0, and let f_1 agree with f_0 except on an input event of per-day probability at most δ, on which it uses E_{out} arbitrarily. For any test ϕ of level α against f_0 on T days,* $\Pr_{f_1}[\phi = 1] \leq \alpha + T\delta$.

Proof sketch. Couple the two pipelines on the same inputs and randomness. Their outputs differ only on the union of the per-day events, which has probability at most $T\delta$. Hence the total-variation distance between the laws of $\mathcal{D}_T$ is at most $T\delta$, and the acceptance probabilities differ by at most that much. □

For a concrete sense of scale, suppose the retry bug fires on one day in a thousand ($\delta = 0.001$) and the audit runs for a 30-day quarter. Then *every* level-0.05 test detects it with probability at most $0.05 + 0.03 = 0.08$, however clever the test. More generally, the auditor cannot distinguish "never leaks" from "leaks rarely enough." The same argument covers a deliberately evasive pipeline, which can make its leaking event as rare as it likes. Black-box auditing is therefore a tool against *bugs*, not adversarial pipelines. Bugs tend to be systematic: a missing filter affects every day and many nodes, which is what makes them detectable. The honest output of an audit is a statement of the form "leakage of magnitude at least ε would have been detected with probability at least $1 - \beta$; none was."

4 Why Naive Tests Fail

Each challenge below starts from a natural *naive test* that an auditor might write first. Section 7.2 measures every one of these failures on compliant pipelines, both in our simulator and on real data. For each one, we give a positive example where the test works, catching a real leak in Figure 1. We then give a counterexample where the same test raises a false alarm on a compliant pipeline, and say what fixes it. Table 2 summarizes the seven challenges.

Challenge	Naive test	Fix
C1 correlation	opted-out pairs close	E_{in} controls, placebo
C2 stochasticity	feature changed	noise-floor control group
C3 rotation	coordinates changed	alignment, invariants
C4 receptive field	1-hop invariance	k-hop invariance
C5 edge identity	untyped invariance	typed edges
C6 opt-out scope	current status on history	fix policy first
C7 dependence, drift	i.i.d. units	customer units

Table 2: Seven naive tests, the challenge that breaks each one, and the fix used in Section 5. Table 5 measures each failure.

C1: Opted-in and opted-out behavior are correlated.
Naive test. Take every pair of items co-purchased by an opted-out customer. Check whether these pairs are closer in feature space than random item pairs. If so, conclude that opted-out baskets shaped the features.
When it works. Suppose the pipeline trains its embeddings on all edges, including Bob's. Bob is the only link between the Grinder and the Tea scoop, so training pulls them together. The test finds them unusually close and correctly flags the leak.
When it fails. Now take a compliant pipeline. Alice bought the Espresso machine and viewed the Grinder, so the pipeline places these two items close together using her edges alone. Bob also bought both, so the pair counts as an opted-out co-purchase, and the test fires anyway. On compliant pipelines, this naive test raises a false alarm on 100% of days in our simulator and on 100% on real data (Table 5).
A stricter version. Keep only pairs that *only* opted-out customers co-purchased, so that no opted-in customer bought both items. This removes the Espresso machine–Grinder pair, and every other pair whose similarity a direct opted-in co-purchase explains. It still fails, with false alarms on 94% of simulated days and 100% of real days, for four reasons.

- *Indirect opted-in paths.* The Grinder and the Tea scoop were never bought together by an opted-in customer. But if Carol bought the Grinder and the Kettle, and Dan bought the Kettle and the Tea scoop, the two items are two hops apart through opted-in edges. Factorizations, GNNs and feature propagation all encode such transitive similarity. In our simulator, 99% of the pairs that pass the stricter filter are still linked by an opted-in path of length two, and these pairs carry all of the residual gap.
- *Shared taste and content.* Bob's basket reflects his preferences, so its items are related in their intrinsic features. A pipeline that feeds content embeddings into F makes them similar even with no opted-in path at all.
- *Selection.* The filter removes most pairs that involve popular items. The surviving pairs are niche and sparse, so they differ from random pairs in degree and in how noisy their

embeddings are.

- *Coverage.* The filter says nothing about aggregates, and it inherits the weakness that two items are linked only if two edges of the same customer leaked.

Fix. A stricter filter on the suspect pairs cannot remove this confounding. What works is a reference group with the same structure: the same tastes, the same indirect paths and the same sparsity, but no possible leak. T5's temporal placebo provides one. Controls computed from E_{in} (T4) are an alternative.

C2: The pipeline is random.

Naive test. Find items whose opted-out edges changed overnight. If any such item's feature also changed, conclude that the pipeline reads opted-out edges.

When it works. Suppose the pipeline is deterministic and its review count wrongly includes opted-out reviews. Bob reviews the Espresso machine on Tuesday, and its review count goes from 0 to 1. The test sees the change and correctly flags the leak.

When it fails. Now take a compliant pipeline that retrains its embeddings daily from a fresh random initialization. Even run twice on the same input, it gives every item a slightly different embedding, the Espresso machine included. The test fires on noise.

Fix. Compare against a control group of items whose inputs did not change, which measures how much features move for no reason (T2). On compliant pipelines, the naive test alarms on 100% of simulated days and 100% of real days, and T2 on 3% and 0% (Table 5).

C3: Embeddings have no fixed coordinates.

Naive test. For items whose inputs did not change, compare yesterday's and today's embeddings coordinate by coordinate, and flag large differences.

When it works. Suppose retraining is warm-started from yesterday's parameters, so the coordinates are stable from day to day. A leak then shows up as a localized coordinate shift near Bob's items, and the test finds it.

When it fails. Now suppose Tuesday's retraining produces exactly Monday's geometry, but rotated. All distances and angles are unchanged, yet every coordinate of every item differs. The test flags the entire catalog.

Fix. Align the two spaces first, e.g., with orthogonal Procrustes (Schönemann, 1966), or compare only quantities that rotations preserve, such as norms and cosine similarities. When embeddings are retrained from a random initialization each day, the naive test alarms on 100% of simulated and real days. Alignment cuts the median relative change of input-unchanged items from 1.40 to 0.014. In the simulator, the rule then never alarms. On real data it still alarms every day, because sparse items keep noisy embeddings (C2). T2 therefore aligns *and* compares against a control group (Table 5).

C4: Features depend on more than immediate neighbors.

Naive test. Restrict attention to items whose opted-in *direct* neighbors are unchanged. Their features should then be unchanged, or move only by noise. If they move more when an opted-out edge changes, conclude leakage.

When it works. For a 1-hop aggregate such as a purchase count, the test is exact. The Grinder's opted-in buyers are unchanged, so its count must be unchanged. If Bob buys a second grinder and the count rises, that is a leak.

When it fails. Now suppose the embeddings come from a two-layer GNN. The Grinder's embedding then depends on its buyers *and on everything those buyers bought*. On Tuesday, Carol buys a new teapot. The Grinder's direct neighbors are unchanged, but its embedding legitimately moves. If such movements happen more often to items with opted-out changes, for example because both are popular, the test blames Bob.

Fix. Check invariance over the pipeline's true receptive field (k hops), which the auditor may have to estimate (T2). A component fit on the whole graph, such as a matrix factorization, has an unbounded receptive field. In our experiments this effect was small. With a 1-hop check, T2's embedding test alarms on 8% of simulated days and 0% of real days, against 5% and 0% with the 2-hop check (Table 5). The changes a 1-hop check misses hit treated and control items alike, so they bias the comparison only when receptive-field churn correlates with treatment. The 2-hop check costs little and removes that risk.

C5: The same pair can change edge type.

Naive test. Represent each item's opted-in neighborhood as the *set of customers* who touched it. If that set is unchanged but a feature changed, conclude leakage.

When it works. If the platform logs a single interaction type, such as purchases only, the customer set fully determines the opted-in input, and the test is sound.

When it fails. With several types, the customer set is not enough. On Monday, Alice viewed the Grinder, and on Tuesday she buys it. The set of customers touching the Grinder is unchanged, but its purchase count correctly rises from 1 to 2. The test sees an unexplained change and blames Bob.

Fix. Track *typed* edges. With untyped invariance, T2's exact aggregate check alarms on 46% of simulated days and 70% of real days, where views often turn into purchases. With typed invariance, it alarms on 0% and 0% (Table 5).

C6: The scope of an opt-out must be defined.

Naive test. Build the expected E_{in} by applying each customer's *current* status to all of their history, then run any of the tests above.

When it works. If the pipeline uses only the current day's interactions, history is irrelevant, and current status is the only sensible definition.

When it fails. Suppose the pipeline computes 30-day purchase counts. Dan opts out on Tuesday. His Monday purchase of the Kettle happened while he was opted in. A pipeline that follows a prospective policy keeps that pur-

chase, which is a defensible choice. The naive test, which assumes a retroactive policy, flags it as a leak. The opposite assumption produces the opposite error.

Fix. Agree on the opt-out policy before testing, and derive E_{in} from that definition. On real data with a 7-day window, a retroactive auditor checking a prospective pipeline alarms on 90% of days. With the agreed policy, it alarms on 0% (Table 5). Our simulator keeps no history, so C6 does not arise there.

C7: Real behavior is dependent and drifts.

Naive test. Treat the units of a test (item pairs in T5, edges in T6, items in T4) as independent. Take the reference from a single side of the current window, such as tomorrow's interactions.

When it works. In our simulator, customers draw their baskets independently from stationary preferences. There, the original T5 and T6 raise false alarms on 2–4% of days, close to the nominal 5%.

When it fails. Real browsing comes in sessions and drifts over time. A customer views several related items in one sitting, so pairs and edges from the same customer are correlated. This week's items differ from next week's, so tomorrow's interactions are not exchangeable with the current window. On RetailRocket, the same tests raise false alarms on 29% (T5) and 13% (T6) of compliant days, and T4 on 17%.

Fix. Use the customer as the unit: at most one pair per customer in T5, and per-customer paired differences in T6. Bracket the window in time: draw the reference from both the interactions that just *left* the window and those that *enter* it next. In T6, also exclude same-day items from each edge's context, so that session coherence cannot favor suspect edges. With these refinements, false alarms fall to 0% and 0% on real data (Section 7.4).

5 Audit Tests

We make one structural assumption, stated for item features; customer and merchant features are analogous.

Assumption 1 (Locality given global state)**.** *There exist a radius $k \geq 1$ and two fixed functions θ and φ, both unknown to the auditor, such that under H_0, for every node v and day t,*

$$F_v^t = \varphi\big(\mathcal{N}_k^{\text{in}}(v),\, x_v,\, \Theta^t,\, \xi_v^t\big), \qquad \Theta^t = \theta(E_{\text{in}}^t, \xi^t). \quad (2)$$

Here $\mathcal{N}_k^{\text{in}}(v)$ is the typed k-hop neighborhood of v in E_{in}^t, Θ^t is a global state such as trained parameters, and ξ_v^t is the part of the pipeline's randomness ξ^t that acts on node v.

Assumption 1 refines the compliant pipeline f_0 in (1) by splitting it into two stages. The *training* stage θ may look at all of E_{in}^t and produces global state Θ^t. The *readout* stage φ computes each node's feature from that state and the node's own k-hop opted-in neighborhood. For a two-layer GraphSAGE model, for example, θ is the training run, Θ^t are the learned weights, φ is the forward pass, and $k = 2$. For a purchase count, Θ^t is empty, φ counts typed edges, and $k = 1$. Since the pipeline is fixed, θ and φ are the same functions every day and for every node. The tests rely on exactly this: in Figure 1, the Tea scoop's feature is $\varphi(\emptyset, x_{\text{scoop}}, \Theta^t, \xi_{\text{scoop}}^t)$, the same function applied to a never-bought item. The assumption rules out no leakage mode. It only says where, under H_0, information may come from. Table 3 summarizes the tests. Each test ends with a short paragraph, *Avoiding the pitfalls*, that explains how its design escapes the naive-test failures of Section 4.

Test	Targets	Needs	Key control
T1 canary	agg., repr.	long tail	empty $\mathcal{N}^{\text{in}}$
T2 zero-diff	agg., repr.	churn	invariant $\mathcal{N}_k^{\text{in}}$
T3 DiD	agg.	status switches	matched items
T4 residual	agg.	—	surrogate of E_{in}
T5 co-occur.	repr.	—	temporal placebo
T6 edge MI	repr.	basket churn	same-user fresh edges
T7 active (ref.)	any	write access	random assignment

Table 3: Audit tests, the leakage modes they target, what they require, and the control that makes them valid under H_0. T1–T6 are passive; T7 is the active reference.

5.1 T1: Natural canaries

T1 uses nodes whose correct output is known in advance.

Natural canary. An item is a *natural canary* on day t if it has at least one opted-out edge and no opted-in edge. In Figure 1, the Tea scoop is a canary, because only Bob bought it. The same idea applies at other levels. Kyoto Crafts is a merchant-level canary, because all of its sales came from opted-out customers. Every opted-out customer, such as Bob, is a customer-level canary.

Cold item. A *cold* item is one that nobody touched on day t, such as a newly listed item.

The test. Under H_0, the pipeline receives exactly the same input for a canary as for a cold item: an empty receptive field. By Assumption 1, both get the feature $\varphi(\emptyset, x_i, \Theta^t, \xi_i^t)$, from the same function. So canaries and cold items with similar intrinsic features must be indistinguishable. In Figure 1, the Tea scoop's purchase count must be 0, and its embedding must look like that of a never-bought tea accessory. Kyoto Crafts must look like a merchant with no sales, and Bob's customer feature must equal the default given to a customer with no history. If the Tea scoop's count is 1, or its embed-

ding sits next to the Grinder (Bob's other purchase), then Bob's edges leaked.

Implementation. We compare canaries with cold items using a permutation test on the feature norm, matching or adjusting on x_i. For customers, the default output is deterministic in our pipelines, so an exact check applies: any opted-out customer whose feature differs from the default is a violation. Long-tailed catalogs supply dozens of new canaries every day: 34 per day in our simulation.

Avoiding the pitfalls. T1 is the most robust passive test, because canaries and cold items receive the same, empty, opted-in input. Correlated behavior (C1) therefore cannot make them differ, and neither can changes elsewhere in the graph (C4), which reach both groups only through the same global state. One residual form of C1 remains: someone *chose* to buy the Tea scoop, so canaries may be intrinsically more attractive than cold items. This is why we match on x_i. Finally, the feature norm is unchanged by rotations of the embedding space (C3), so no alignment is needed.

5.2 T2: Zero-difference on invariant neighborhoods

T2 compares how much features move overnight for two groups of items whose opted-in inputs did not change. It needs two definitions.

Receptive field. Under Assumption 1, the only opted-in data that can affect F_v^t *locally* is $\mathcal{N}_k^{\text{in}}(v)$: the typed opted-in edges within k hops of v. Everything else reaches v only through the global state Θ^t. We call $\mathcal{N}_k^{\text{in}}(v)$ the *receptive field* of v. In Figure 1, suppose the pipeline is a two-layer GNN ($k = 2$). The Grinder's receptive field then contains its own opted-in edges (Alice's view, Carol's purchase) and the other opted-in edges of those customers (Alice's purchase of the Espresso machine, Carol's purchase of the Kettle). Dan's purchase of the Kettle is three hops away and lies outside it. For a purchase count ($k = 1$), the receptive field is just the item's own opted-in edges.

Opted-out neighborhood. Let $\mathcal{N}^{\text{out}}(v)$ be the set of typed opted-out edges incident to v. The Espresso machine's opted-out neighborhood is $\{(\text{Bob}, \text{buy})\}$. It *changes* between two days when an opted-out customer adds, removes or changes the type of an interaction with the item. If Bob also reviews the Espresso machine on Tuesday, its opted-out neighborhood becomes $\{(\text{Bob}, \text{buy}), (\text{Bob}, \text{review})\}$.

The test. Select items whose receptive field is identical on days t and $t{+}1$. Under H_0, the features of these items can still move, but only because Θ was retrained and because of noise. Those causes affect all selected items alike. Split the selected items into two groups. *Treated* items have a changed opted-out neighborhood, and *control* items do not. A compliant pipeline never reads the opted-out edges, so treated items should move no more than control items. In Figure 1, if Bob reviews the Espresso machine on Tuesday and nothing in its receptive field changes, the Espresso machine is treated. If its embedding moves noticeably more than those of comparable control items, Bob's review leaked.

To implement the test, we align the two days' embeddings, compute each item's relative change $\|\Delta F_i\|/\|F_i\|$, and regress its logarithm on the treatment indicator and log degree. We then test whether the treatment coefficient is positive. For a deterministic 1-hop aggregate, no regression is needed: any change in a treated item's count is a violation, which gives an exact test. Treatment is defined by the item's *own* opted-out edges, because those are the ones a leak affects most. Opted-out changes farther away may also move control items under a leak. That dilutes the contrast and costs power, but it cannot cause false alarms.

Avoiding the pitfalls. The control group measures the noise floor, so random retraining (C2) moves both groups alike. Embeddings are aligned before comparison (C3). Invariance is checked over the full receptive field, not just direct neighbors (C4), and on typed edges (C5). Finally, treated and control items have identical opted-in inputs on both days, so correlated behavior (C1) cannot separate them.

5.3 T3: Status-switch difference-in-differences

T3 checks that a customer's contribution disappears, or appears, on the day their status changes.

Status switch. A customer *switches* when their opt-out status changes between days t and $t{+}1$. In Figure 1, Dan opts out on Tuesday. Under a retroactive policy (the choice discussed in C6), all of Dan's edges move from E_{in} to E_{out} on that day. Call the items Dan touched *affected* items. Here, the Kettle is affected.

The reference change, without a formula. The auditor cannot compute how the Kettle's feature *should* change, because it does not know the pipeline. A compliant pipeline, however, sees Dan's switch exactly as if Dan had deleted his interactions: one opted-in purchase disappears from the Kettle. Items lose opted-in edges for ordinary reasons every day, for instance when a customer's basket changes. These *natural removals* show, empirically, how the pipeline responds to losing an edge of a given type. For each affected item, we therefore select *comparison* items that lost the same number and types of opted-in edges that day for ordinary reasons,

matched on degree. Suppose the Toaster lost one purchase on Tuesday because its buyer's basket changed. A compliant pipeline must treat the Kettle and the Toaster alike: whatever happens to the Toaster's purchase count and embedding on Tuesday should also happen to the Kettle's. Switches back to opt-in work the same way, with *natural additions* as the reference: when Bob opts back in, his items should respond like items that gained the same opted-in edges for ordinary reasons. With many days of data, the auditor can go further and fit a model of the pipeline's response, ΔF_i as a function of the change in item i's typed opted-in edge counts, on items that no switch affected, and use it to predict the affected items' change.

The test. We compare affected and comparison items in a difference-in-differences design (Angrist and Pischke, 2009), using the change from t to $t{+}1$ and on the following days. Compliance predicts that affected items respond on the switch day exactly as comparison items do, and that neither group shows further change afterwards. Two bugs produce different patterns:

- *Stale cache or delayed deletion.* On Tuesday, the Toaster's count drops but the Kettle's does not, because the pipeline still uses Dan's purchase. The Kettle catches up only on Thursday. Affected items respond *less* than comparison items on the switch day, and *more* on later days.
- *Opted-out data used before an opt-in.* Bob opts back in on Thursday, so his edges legitimately join E_{in} on Thursday. If his items already carried his signal on Wednesday, they respond *less* than comparison items on Thursday, because there is nothing left to add.

For deterministic columns, such as counts, the comparison items also reveal how the column responds to one removed edge. If every comparison item's count drops by one, a single affected item whose count does not drop is a violation.

Avoiding the pitfalls. Comparison items have the same kind of opted-in change on the same day, so correlated behavior (C1) and retraining noise (C2) affect both groups alike. Embeddings are aligned before comparison (C3). Edge changes are matched on type (C5), and the opt-out policy is fixed in advance (C6).

What it adds. T3 targets how status changes propagate through the pipeline: stale consent tables, caches and delayed deletions. Such bugs can pass T1 and T2 on days without switches. Our simulated pipelines have no caching, so we do not evaluate T3 in Section 7.

5.4 T4: Residual regression with surrogate controls

T4 asks whether F knows something about opted-out edges that E_{in} cannot explain.

Opted-out summary. For each item, let $S_i^{\text{out}} = \log(1 + |\mathcal{N}^{\text{out}}(i)|)$ summarize its opted-out edges. In Figure 1, $|\mathcal{N}^{\text{out}}(i)|$ is 1 for the Espresso machine, the Grinder and the Tea scoop, and 0 for the Kettle.

Surrogate features. The auditor computes its own features Z_i from E_{in} and x alone: typed counts, its own factorization of E_{in}, and a propagation of x over E_{in}. For the Grinder, Z encodes that Alice viewed it, that Carol bought it, and that it resembles the Espresso machine.

The test. Fit two predictors of S^{out}. Model A uses Z only, and model B uses Z and F. Under H_0, F is a function of E_{in} plus noise. If Z captures what E_{in} says about S^{out}, model B cannot beat model A on held-out items. If it does, F carries opted-out information. For example, suppose the pipeline reports a purchase count of 2 for the Grinder, while the auditor's count from E_{in} is 1. Model B then learns that the difference between the two counts predicts S^{out} exactly.

Implementation. We use ridge regression with 5-fold cross-fitting, in the spirit of double machine learning (Chernozhukov et al., 2018). We compute the per-item difference in held-out squared error between the two models and apply a one-sided t-test. It suits aggregates best. Against representation leakage it is weak, because a count of opted-out edges discards *which* edges leaked.

Avoiding the pitfalls. The surrogate Z is what protects T4 against correlated behavior (C1). Model A already uses everything Z knows about E_{in}, so model B can only win with information that Z lacks. The protection is only as good as Z. If F encodes opted-in information that Z misses, false alarms are possible. In our experiments the test is conservative. Both models predict a per-item target from a single day's features, so randomness (C2) and rotations (C3) act as noise, not bias.

5.5 T5: Co-occurrence membership inference with a temporal placebo

T5 targets representation leakage. Training on Bob's basket pulls together the items Bob bought together, much as membership-inference attacks detect training data (Shokri et al., 2017; Carlini et al., 2022). On graphs, the natural unit is a co-interaction (He et al., 2021; Olatunji, Nejdl, and Khosla, 2021).

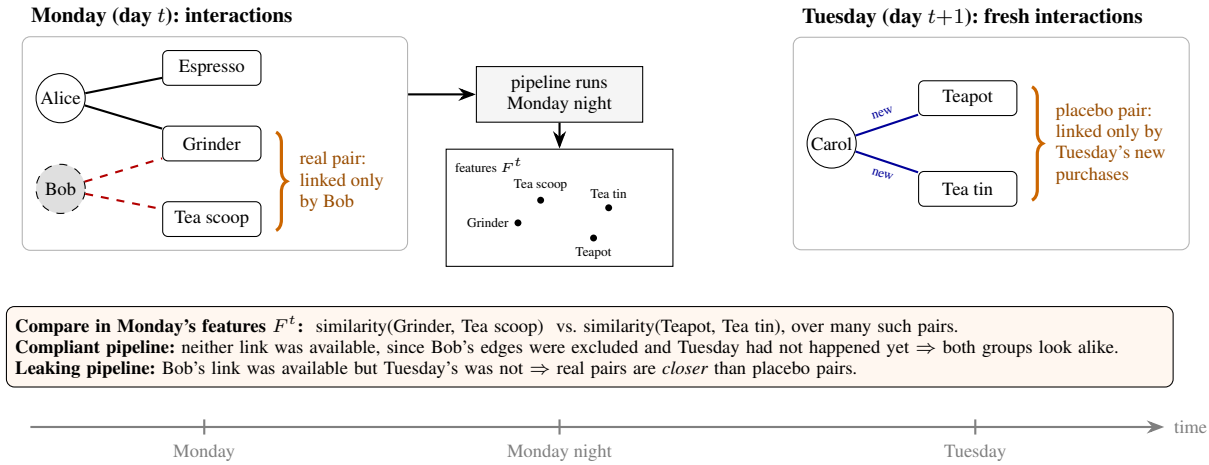


Figure 2: The temporal placebo of T5. Real pairs are linked only through opted-out customers on day t. Placebo pairs are linked only through interactions that first appear on day $t+1$, after day t's features were computed. Both kinds of pair arise from the same shopping behavior, so they share any taste-driven similarity. Only the real pairs could have leaked.

Real pairs. Let $\mathcal{R}^t$ contain the item pairs that are co-purchased *only* through opted-out customers on day t, and never together in E^t_{in}. In Figure 1, (Grinder, Tea scoop) is a real pair, because only Bob bought both.

Placebo pairs. Comparing real pairs with random item pairs is the naive test of C1, and it fails because shared taste makes real pairs similar anyway. We therefore need a reference group with the same taste-driven similarity but no possible leak. Let $\mathcal{Q}^t$ contain the pairs that co-occur only through interactions that first appear on day $t+1$, which the day-t pipeline could not have seen. For example, if Carol buys a teapot and a tea tin together on Tuesday, then (teapot, tea tin) is a placebo pair for Monday. Placebo pairs come from the same behavior that produces real pairs, so they share the taste-driven similarity that broke the naive test, but they cannot have leaked into Monday's features. Figure 2 shows the timeline.

The test. Compare the cosine similarity of real and placebo pairs in F^t. Under H_0 the two distributions are the same. If real pairs are systematically closer, the pipeline learned from opted-out baskets.

Implementation. Two details are essential. First, the two groups must use symmetric exclusions. Neither may co-occur in E^t_{in}. Real pairs exclude pairs that also co-occur in day $t+1$'s fresh interactions, and placebo pairs exclude pairs that also co-occur in E^t_{out}. Second, overlapping pairs share items and are therefore dependent, which makes standard errors too small. We subsample pairs so that no item appears twice. We then regress similarity on group membership and the two items' log degrees, and test the group coefficient. Section 7 shows that both details are necessary. Real data needs two further refinements (C7). First, each customer contributes at most one pair, because pairs from one browsing session are correlated. Second, the placebo brackets the window in time. It combines pairs that co-occur only through interactions that *left* the window on day t with pairs that co-occur only through interactions that *enter* it on day $t+1$.

Avoiding the pitfalls. The placebo absorbs correlated behavior (C1). Both groups are scored with the same day's features, so randomness (C2) affects them alike. Cosine similarity within one embedding space is unchanged by rotations (C3).

5.6 T6: Edge-level membership inference

T6 asks directly whether the pipeline *remembers* individual opted-out edges. It applies the logic of membership inference: a trained model tends to score its training data higher than data it never saw (Shokri et al., 2017; Carlini et al., 2022).

Scoring an edge. For an edge (u, i) on day t, the auditor builds a context vector for customer u from F alone: the mean embedding of u's other items that day. The edge's score is the cosine similarity between item i's embedding and this context. In Figure 1, Bob's edge to the Tea scoop is scored by comparing the Tea scoop's embedding with the mean embedding of the Espresso machine and the Grinder. The context is needed because a compliant pipeline gives Bob only the default customer feature (T1).

Three groups of edges.

- *Members* (IN): opted-in edges on day t. The pipeline legitimately learns from them, so they score high if it memorizes edges.
- *Suspects* (OUT): opted-out edges on day t, such as Bob's three purchases.
- *Guaranteed non-members* (FRESH): for opted-out customers whose basket changes on day $t{+}1$, their day-$(t{+}1)$ basket, scored with day-t features. The day-t pipeline cannot have seen these interactions.

The test. We compare OUT with FRESH, not with IN. Under H_0, both are non-members. They come from the same customers with the same tastes, so they score alike. Under a leak, OUT edges become members and score higher. For each opted-out customer with both kinds of edge, we compute the difference between the mean OUT score and the mean FRESH score. We then test whether these differences are positive across customers (one-sided t-test). Comparing OUT with IN directly would not work. How much higher members score than non-members depends on how much the pipeline memorizes. A pipeline that generalizes smoothly scores members and non-members alike even when it is compliant, so "OUT looks like IN" would not imply a leak. FRESH supplies the missing reference.

An effect size. IN provides a scale. Let INF be the next-day baskets of opted-in customers, scored the same way, and let $\bar{s}$ denote a group's mean score. Define

$$\lambda = \frac{\bar{s}_{\mathrm{OUT}} - \bar{s}_{\mathrm{FRESH}}}{\bar{s}_{\mathrm{IN}} - \bar{s}_{\mathrm{INF}}}. \tag{3}$$

$\lambda \approx 0$ under compliance, and $\lambda \approx 1$ when the pipeline treats all opted-out edges as training data. In between, λ roughly estimates the fraction of opted-out edges that behave like members. The denominator doubles as a power diagnostic. If $\bar{s}_{\mathrm{IN}} \approx \bar{s}_{\mathrm{INF}}$, the pipeline does not memorize individual edges, T6 cannot detect anything, and the audit report should say so.

Relation to T5. T5 tests item pairs, and T6 tests edges, so the two are different tests. They are nested, though: an edge's score pools all the pairs that pass through customer u and end at item i. The Tea scoop's score aggregates its pairs with the Espresso machine and with the Grinder. The references also differ. T5's placebo pairs come from anyone's fresh interactions, while T6's reference comes from the same customers, which also controls for who opts out. Both tests link two items only if two edges of the same customer leaked. They are therefore weak when individual edges leak at random, since both edges leak with probability ρ^2. They are strong when a bug leaks whole customers, for example through stale consent rows.

Implementation. Two details nearly broke calibration in our experiments. First, the reference must be the customer's *whole* next-day basket, not only the new edges. New edges exclude items the customer already had, so they are biased away from the customer's strongest preferences, as with T5's placebo. Second, both groups' contexts must have the same size. We drop one random item from the context of each reference edge. We also tried a learned scorer: a logistic regression trained on opted-in customers to separate IN from INF, whose labels are free, and then applied to opted-out customers. It must use only features derived from F. Features computed from the edge lists, such as an item's degree, include the edge itself and so encode the group by construction. With them, the learned scorer rejected H_0 on 75% of clean days. With F-derived features only, it is calibrated, but in our simulator it does not beat the cosine score. On real data, two refinements are needed (C7). The reference also includes the customer's edges that just left the window, which brackets it in time. Each suspect edge's context also excludes items from the same day, so that session coherence cannot favor suspect edges.

Avoiding the pitfalls. OUT and FRESH come from the same customers, so correlated behavior (C1) cancels. Both groups are scored with the same day's features, so retraining noise (C2) affects them alike, and cosine similarity within one embedding space is unchanged by rotations (C3).

5.7 T7: Active canaries as a reference test

T1–T6 are *passive*: they use only data the platform produces anyway. If the auditor can write to the system, it can create its own Bobs, whose behavior it controls. T7 does this. Because its validity rests on the auditor's own randomization, we use it as a reference against which the passive tests are measured.

Design. Each day, the auditor enrolls C synthetic opted-out accounts. It draws $2Cm$ items that no real customer touched that day, and splits them uniformly at random into a treated half and a control half. Each canary account purchases m treated items. In Figure 1, a canary account "Eve" might buy four never-touched items, chosen by a coin flip only the auditor knows.

The test. Under H_0, the pipeline never reads the canaries' edges. Treated and control items are then exchangeable *by construction*, and a permutation test over the random split is exact. It needs no assumption about how the pipeline works or how real users behave. We add the exact customer-level check from T1: the canary accounts' own features must equal the default.

Avoiding the pitfalls. None of the pitfalls of Section 4 applies. Random assignment breaks any link between treatment and behavior (C1). Noise and rotations affect treated and control items alike (C2, C3). No locality assumption is needed (C4).

Role as a baseline. T7's validity is guaranteed and its power is set by the canary budget, so it serves as a yardstick in two ways. In Section 7, it measures how much power the passive tests achieve without write access. In deployment, a small periodic canary campaign checks the passive tests on the real pipeline. If T7 detects a leak that T1–T6 miss, the passive tests are blind to that leakage mode, and the audit's reported bounds must say so.

Costs. T7 requires write access, which an external assessor may lack. Its power grows with the number of canary edges. Canary accounts should touch only cold items and stay opted out, so that a compliant pipeline ignores them and real users are unaffected. They must also be indistinguishable from real accounts to the pipeline's owners. Otherwise, a pipeline could special-case them and evade the test.

6 Sequential Evidence and Reporting

The tests of Section 5 each produce one p-value per day. This section answers two practical questions. First, given an endless stream of daily p-values, when should the auditor raise an alarm? Second, when no alarm is raised, what should the audit report?

The problem with daily p-values. The obvious rule is to raise an alarm on the first day with $p < 0.05$. Each day, a compliant pipeline then triggers a false alarm with probability 5%. Over a 30-day audit with independent days, the chance of at least one false alarm is $1 - 0.95^{30} \approx 79\%$. A Bonferroni correction (alarm only if $p < 0.05/30$) restores the error rate, but it fixes the audit length in advance and throws away evidence that accumulates over several days.

E-values as bets. E-values solve both problems (Shafer et al., 2011; Vovk and Wang, 2021; Ramdas et al., 2023). Think of the auditor as betting against compliance, starting with \$1. Each day it converts the p-value into a multiplier $e_t = 1/(2\sqrt{p_t})$ and multiplies its wealth: $M_T = \prod_{t \le T} e_t$. Under H_0 the p-value is uniform, and the expected multiplier is exactly 1, so the bet is fair and wealth does not grow on average. The auditor raises an alarm when wealth reaches $1/\alpha = 20$. A fair game that starts at 1 reaches 20 with probability at most 1/20 (Ville's inequality). The false-alarm rate is therefore at most 5% *overall*, however long the auditor monitors and whenever it stops. Under a leak, small p-values give large multipliers, and wealth grows quickly.

Day t	1	2	3	4	5
p-value p_t	0.40	0.03	0.20	0.01	0.02
multiplier $e_t = 1/(2\sqrt{p_t})$	0.79	2.89	1.12	5.00	3.54
wealth M_t	0.79	2.28	2.55	12.8	**45.1**

Table 4: Accumulating daily evidence. The alarm threshold is 20, and it is crossed on day 5.

Table 4 gives a worked example. No single day is below the Bonferroni threshold $0.05/30 \approx 0.0017$, yet moderate evidence on several days compounds into an alarm on day 5. Under compliance, p-values are typically large: $p = 0.6$ gives $e = 0.65$, and wealth shrinks. An exact test that observes a violation returns $p = 0$, which raises an alarm at once.

Combining tests. To run several tests at once, the auditor splits its \$1 equally across them and raises an alarm when the *average* wealth reaches 20. An average of e-values is itself an e-value, so no multiplicity correction is needed. A test that has no power against a given pipeline simply loses its share.

Caveat: dependence across days. Ville's inequality requires each day's e-value to be valid given all earlier days. Daily graphs share customers, so this is an assumption rather than a guarantee. It becomes more plausible when each day's test uses fresh units, for example new canaries, or splits customers across days.

What to report when there is no alarm. Proposition 1 rules out the claim "no leakage." The audit should instead report a *detection bound*: "had the pipeline leaked a fraction ρ of opted-out edges, an alarm would have been raised within T days with probability at least $1 - \beta$." Computing this bound requires a version of the pipeline into which leaks can be injected. With write access, T7 provides it on the real pipeline. Without write access, the auditor builds a surrogate pipeline on the observed data and injects leaks there, as we do in Section 7. Using those results as an illustration, a final report would read: *"No leakage was detected. Leakage of 2% or more of opted-out edges into aggregates would have been detected within one day. Leakage of 10% or more into embeddings would have been detected within four days. Smaller leakage into embeddings cannot be ruled out."* In our simulation, every audit with these leak rates raised an alarm within those horizons. A real report would use enough simulated audits to state the probability $1 - \beta$ precisely.

7 Experiments

We evaluate in two settings. In a synthetic simulator we control every part of the ground truth. On real interaction data

from RetailRocket, behavior is realistic, while opt-outs and leaks are simulated.

7.1 Synthetic setup

Simulator. We simulate 3,000 customers, 2,000 items with intrinsic embeddings $x_i \in \mathbb{R}^{16}$ and Zipf popularity (exponent 1.1), and 100 merchants. Each customer draws $1 + \mathrm{Poisson}(7)$ items with probability proportional to popularity times $\exp(2\, p_u^\top x_i/4)$. Edge types are view, save, purchase and review, with probabilities $0.60/0.15/0.17/0.08$. Each day, 20% of customers redraw their interactions. Opt-out status follows a two-state Markov chain with stationary rate 10% (in→out 0.005, out→in 0.045). Each audit spans 10 days.

Pipelines. Item features concatenate (i) typed counts and the number of unique customers (aggregation component) with (ii) a rank-16 randomized SVD of the type-weighted interaction matrix (Halko, Martinsson, and Tropp, 2011) and (iii) a propagation of x over the item–item co-interaction graph, in the style of SIGN (Frasca et al., 2020) (representation component). Customer features are typed counts and the mean embedding of their items. The *clean* pipeline uses E_{in} everywhere. The *aggregation-leak* pipeline adds a fraction ρ of opted-out edges, selected by a fixed hash, to (i) and customer pooling only. The *representation-leak* pipeline adds them to (ii) and (iii) only. The SVD is reseeded daily, so all pipelines are stochastic.

Leak patterns. We choose which opted-out edges leak in two ways, with the same expected fraction ρ. In *per-edge* leakage, each opted-out edge leaks independently, selected by a fixed hash of the edge. In *per-customer* leakage, a fraction ρ of opted-out customers leak all of their edges, as when the consent table is stale for some customers. Real bugs are more often of the second kind, or leak a whole interaction type at once.

Tests. We run T1, T2 (k=1 for aggregates and k=2 for embeddings), T4 (ridge, 5-fold cross-fitting, with surrogate Z = typed log-counts, x, a rank-32 SVD of E_{in} and a propagation over E_{in}), T5 and T6. For T6 we report the cosine score, and we also ran the learned scorer. Our pipelines have no caching, so T3 adds nothing here and we omit it. T7 runs in a separate simulation with the same seeds. It adds $C = 20$ canary accounts that each buy $m = 4$ items. That gives 80 treated items, 80 control items and 80 canary edges per day, about 3% of the roughly 2,400 organic opted-out edges. A typical day yields 34 canaries against 210 cold items, 40 treated and 406 control items for T2, 298 real against 412 placebo item-disjoint pairs for T5, and about 60 opted-out customers with both suspect and reference edges for T6.

		Naive		Fixed	
	Naive test → fix	Sim.	Real	Sim.	Real
C1	opted-out pairs vs. random → T5	100	100	2	0[†]
	only-opted-out pairs vs. random	94	100		
C2	feature changed → T2	100	100	3	0
C3	raw coordinates → aligned	100	100	0	100[‡]
C4	1-hop → 2-hop invariance	8	0	5	0
C5	untyped → typed invariance	46	70	0	0
C6	retroactive auditor → agreed policy	–	90	–	0
C7	basic → refined T5 / T6	2 / 4	29 / 13	7.5 / 5	0 / 0

Table 5: False-alarm rate (% of days, $\alpha = 0.05$) of each naive test and its fix on compliant pipelines: 300 simulated and 40 real days (200 and 56–98 for C7). [†]Refined T5 (C7); the basic T5 alarms on 29% of real days. [‡]Alignment removes the rotation (the median relative change falls from 1.40 to 0.014), but sparse real items keep a heavy tail of unstable embeddings, which only T2's control group absorbs (0%).

7.2 Naive tests fail

Table 5 runs each naive test of Section 4, and its fix, on compliant pipelines, so every alarm is a false alarm. The real-data columns use the RetailRocket setup of Section 7.4. For C3 we rotate the embedding block by a random orthogonal matrix each day, to emulate retraining from a random initialization. All of our tests are invariant to this rotation.

Four findings stand out.

- C1, C2, C3, C5 and C6 break their naive tests on almost every day, on both data sources, and the fixes restore calibration. The stricter "only" filter of C1 barely helps: 94% and 100%.
- C4 did not cause false alarms in our pipelines. The 2-hop changes that a 1-hop check misses affect treated and control items alike, so they bias T2 only when receptive-field churn correlates with treatment. We keep the 2-hop check because it is cheap and safe.
- On real data, alignment alone does not rescue the coordinate rule of C3. It removes the rotation, but not the noise of sparse items. T2 handles both.
- C7 appears only on real data, where the basic T5 and T6 alarm on 29% and 13% of compliant days. The refined versions alarm on none.

7.3 Calibration and power on synthetic data

Calibration. Over 30 clean audits (300 test-days), the per-day rejection rates at $\alpha = 0.05$ are 1% (T1), 3% (T2), 0% (T4), 2% (T5), 4% (T6; 6% with the learned scorer) and 0% (T7). No e-process, individual or averaged, crosses $1/\alpha = 20$ in any audit.

Beyond the naive tests of Section 7.2, several design choices that look reasonable failed in pilot runs. Building T5's placebo from only the *new* edges of day t+1, with pair-level OLS errors, yields **18–20%**. New edges are biased

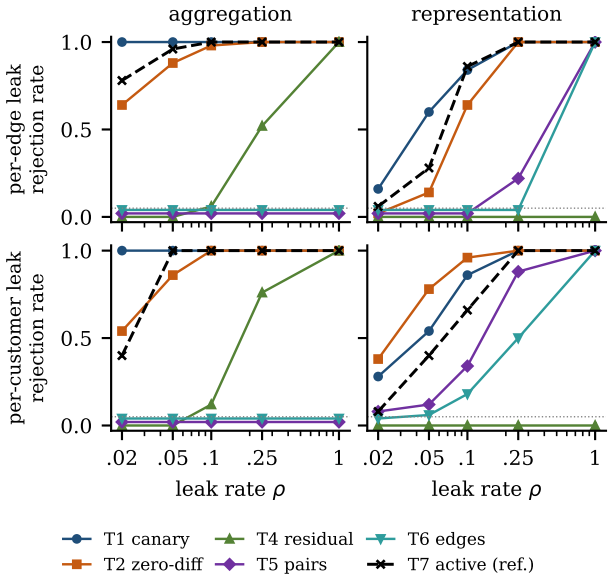


Figure 3: Per-day rejection rate ($\alpha = 0.05$) against the fraction ρ of opted-out edges leaked, averaged over 5 seeds $\times$ 10 days. Rows: leak pattern. Columns: leakage channel. Dotted line: nominal level. Dashed black: the T7 reference with 20 canary accounts and 80 canary edges per day. T4 targets aggregates only; T5 and T6 target representation leakage only.

	leak rate ρ				
Leak / channel / tests	.02	.05	.10	.25	1.0
Per-edge leaks					
Aggregation, passive	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)
Aggregation, T7	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)
Representation, passive	0/5	3/5 (3)	5/5 (2)	5/5 (1)	5/5 (1)
Representation, T7	0/5	4/5 (4.5)	5/5 (2)	5/5 (2)	5/5 (2)
Per-customer leaks					
Aggregation, passive	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)
Aggregation, T7	2/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)	5/5 (1)
Representation, passive	5/5 (7)	5/5 (2)	5/5 (1)	5/5 (1)	5/5 (1)
Representation, T7	2/5 (5.5)	5/5 (5)	5/5 (2)	5/5 (2)	5/5 (2)
Clean, passive and T7	0/30 false alarms each				

Table 6: Audits in which the e-process reached 20 within 10 days (median detection day in parentheses). "Passive" averages the e-processes of T1, T2, T4, T5 and T6; T7 uses 20 canary accounts and 80 canary edges per day.

away from items a customer already had, and overlapping pairs violate independence. T6 repeated both lessons in pilot runs. A learned scorer that also used item degree and context size rejected H_0 on 75% of clean days, because these features differ between current and next-day edges by construction. With F-derived features only, but with only the *new* next-day edges as the reference, it still rejected on 44%. All of these failures are confounding, not lack of power, and none is visible without a null study.

Power. Figure 3 and Table 6 summarize the results.

Per-edge leaks, aggregation. T1 alone rejects on every day even at $\rho = 0.02$, which leaks roughly 0.2% of all edges. An opted-out customer with any leaked edge has a non-default feature, and the exact check catches it. T2's exact aggregate check follows closely. T4 needs $\rho \geq 0.25$, because summarizing E_{out} by degree throws away the identity of the leaked edges.

Per-edge leaks, representation are harder. The combined passive e-process detects them by day 2 (median) at $\rho = 0.1$, and in 3 of 5 audits at $\rho = 0.05$. T1 is again the most powerful single test: the factorization assigns a nonzero embedding to canary items whose training column is no longer empty. T5 detects only $\rho \geq 0.25$, and T6 only $\rho = 1$. This is the ρ^2 effect predicted in Section 5: a link between two items needs two leaked edges of the same customer.

Per-customer leaks change the picture for the co-interaction tests. When whole customers leak, their edges leak together, so T5 rejects on 88% of days and T6 on 50% at $\rho = 0.25$, against 22% and 4% for per-edge leaks. T6's effect size λ now tracks the leak rate: its median is 0.29 at $\rho = 0.25$ and 0.98 at $\rho = 1$. It is biased upward by about 0.09, its median on clean days, so it is informative only well above that level. The combined passive e-process detects every per-customer leak, including $\rho = 0.02$ in the representation, where the median detection day is 7. The learned T6 scorer behaves like the cosine score throughout. Its per-day rejection rates differ by at most 0.06.

Passive tests against the T7 reference. T7 is exactly valid, with zero false alarms, and detects every leak with $\rho \geq 0.1$. The passive T1 matches or beats it in every setting except one: per-edge representation leakage at $\rho = 0.1$, where T7 rejects on 86% of days and T1 on 84%. For per-edge leaks, the reason is budget: natural opt-outs supply about 30 times more "canary edges" per day than our 80 synthetic ones, at no cost. For per-customer leaks, the gap is starker, because T7 becomes a lottery. With 20 canary accounts and $\rho = 0.02$, the chance that any canary account leaks is $1 - 0.98^{20} \approx 33\%$, and T7 detects only 2 of 5 audits. The roughly 300 natural opted-out customers contain about 6 leaking customers on any day. Two lessons follow. First, a passive audit built on natural canaries can be as powerful as a modest active campaign, without write access. Under per-customer leakage, an active campaign should spend its budget on many accounts rather than many edges per account. Second, T7's value lies in its guaranteed validity, not its power. It checks the passive tests on the real pipeline, where their assumptions cannot be verified directly.

	Clean	Aggregation		Representation		
Test	FPR	.1	1.0	.1	.1 cust.	1.0
T1	0	100	100	100	–	100
T2	0	100	100	0	–	100
T4*	17	100	100	0	–	15
T5 basic*	29	–	–	60	85	100
T5 refined	0	–	–	0	5	100
T6 basic*	13	–	–	10	15	90
T6 refined	0	–	–	0	5	100

Table 7: RetailRocket, per-day rejection rate (%). Clean: false-alarm rate on compliant pipelines (42 days for T1, T2 and T4; 98 for basic T5 and T6; 56 for refined). Leak columns give the leak rate ρ, with per-edge leakage unless marked "cust."; 2 opt-out draws $\times$ 10 days. *Not calibrated on real data, so its power is not meaningful.

7.4 Real data: RetailRocket

Data. We use the public RetailRocket e-commerce event log (RetailRocket, 2016). It covers 4.5 months and contains 2.66M views, 69k add-to-carts and 22k transactions by 1.4M visitors on 235k items. We drop visitors with fewer than 2 or more than 200 events. Single events carry no co-interaction, and the heaviest visitors are mostly crawlers. That leaves 405,832 customers and 162,225 items. The log has no merchant field, so our real-data audits cover customers and items only.

Semi-synthetic design. The interaction graph is real. Opt-outs and leaks are simulated, because real consent labels are not public. The snapshot for day t contains the typed edges of the window $[t-6, t]$, one per (customer, item) pair with the strongest event type, about 60k edges per day. Intrinsic item embeddings come from an SVD of days 0–29, which no audit window includes. Opt-out status, pipelines and leak patterns are as in the simulator. Each audit covers 10–14 days, starting on day 37.

Structural tests hold. T1 and T2 raise no false alarm on 42 compliant days (Table 7). The real long tail supplies about 1,900 natural canary items per day, against 34 in the simulator. As a result, T1 rejects on every day for every leak in our grid, including per-edge representation leakage at $\rho = 0.1$, where every other calibrated test is blind. Its e-process raises an alarm on day 1 for aggregation leaks and on day 2 for representation leaks. With 2,000 permutations, a single day's p-value cannot fall below $1/2001$, so one day of evidence falls just short of the threshold. More permutations would bring the alarm to day 1.

Behavioral tests need the refinements. The basic T4, T5 and T6 rely on exchangeable behavior, and they alarm on 17%, 29% and 13% of compliant days (C7). The refined T5 and T6 raise no false alarm on 56 compliant days. They keep full power at $\rho = 1$, but detect little at $\rho = 0.1$ (0–5% of days). On synthetic data, the refined T5 alarms on 7.5% of 200 compliant days, slightly above the nominal 5%, and the refined T6 on 5%. There, the refined T6 is more powerful than the basic one under per-customer leakage (70% against 48% of days at $\rho = 0.25$), and the refined T5 is less powerful (56% against 74%). T4 remains miscalibrated on real data. Items that share customers are dependent, and a cluster-robust version is future work. We therefore exclude T4 from real-data alarms.

Takeaway. Real behavior separates the tests into two groups. Tests built on structural facts, an empty or unchanged receptive field (T1, T2), carry over from the simulator unchanged and stay powerful. Tests built on behavioral exchangeability (T4, T5, T6) are sound in the simulator but need cluster-aware units and time-bracketed references on real data. A null study on real data is therefore not optional. It is what revealed C7.

8 Related Work

To our knowledge, no prior work audits opt-out compliance of a feature pipeline from its inputs and outputs alone. Several lines of work come close, and each differs in a way that matters for our setting.

Auditing consent compliance. Liu, Iqbal, and Saxena (2024) audit whether advertisers honor opt-outs signaled through consent management platforms. They infer data use from advertisers' bidding behavior and find that several ad platforms ignore consent. Their auditor, like ours, sees only external behavior. The object is different, though. They measure whether third parties collect and share data, not how a first party's derived features depend on opted-out records. From the legal side, Biega and Finck (2021) analyze purpose limitation in data-driven systems and note that black-box methods for auditing such compliance had not been developed.

Data-use auditing and membership inference. Membership inference asks whether a record was in a model's training set (Shokri et al., 2017; Carlini et al., 2022). For recommender systems, Zhang et al. (2021) infer whether a user's history was used from the recommendations alone. Huang, Gong, and Reiter (2024) turn membership inference into *proactive* data-use auditing. A data owner publishes one of two marked versions of each record and applies a sequential test with a controlled false-detection rate. These works audit a single trained model with query access, usually for data the auditor itself marked. Our T6 and T7 borrow these ideas. The passive tests (T1–T6) instead exploit natural experiments in continuously recomputed graph features, where

removal must also propagate to neighbors and the main obstacle is confounding by correlated behavior (Section 4).

Verifying unlearning. Athena (Sommer et al., 2022) verifies deletion requests by having some users plant backdoors in their data and testing whether the backdoor survives. Eisenhofer et al. (2025) require the server to produce cryptographic proofs of training and unlearning. Thudi et al. (2022) show that the model parameters alone cannot certify that a point was unlearned, which echoes our Proposition 1. Unlearning methods (Ginart et al., 2019; Bourtoule et al., 2021), including graph and recommendation variants (Chen et al., 2022a,b), make removal efficient, but they are mechanisms, not audits. Our setting differs in three ways. Opt-outs are continuous rather than one-off requests, the auditor has no query access to a model, and the influence of removed data spreads through the graph.

Privacy auditing. Empirical DP auditing inserts canaries and measures distinguishability to lower-bound privacy loss (Jagielski, Ullman, and Oprea, 2020; Nasr et al., 2021; Steinke, Nasr, and Jagielski, 2023). Our tests can be read as auditing a degenerate mechanism that should have $\varepsilon = 0$ with respect to E_{out}. The auditor usually cannot choose the inputs, which is why we rely on natural experiments, and the reference distribution must be built from correlated behavioral data rather than from a known mechanism.

Enforcement inside the system. Industry also enforces consent and purpose limitation from the inside. Meta's Policy Zones, for example, attach purpose annotations to data and track information flow at runtime across its warehouse and serving stacks (Meta, 2024). Such white-box controls and our black-box audit complement each other. Information-flow control prevents violations where it is deployed. Output-based auditing checks the result independently, including components that the enforcement layer does not cover.

Graph privacy attacks. Link stealing and GNN membership inference (He et al., 2021; Olatunji, Nejdl, and Khosla, 2021) show that node outputs reveal edges. T5 and T6 turn these attacks into audit statistics, with references that separate memorization from homophily.

9 Limitations

Our real-data study is semi-synthetic: the interactions are real, but opt-outs and leaks are simulated, and RetailRocket has no merchant layer. Real opt-out decisions may correlate with behavior. Privacy-conscious customers may browse differently, which would add confounding beyond C7. Before deployment, every test should first be run on days or slices where compliance is known, such as a pipeline replica fed only E_{in}. That is how we found C7. T4 is not calibrated on real data, and the refined T5 is slightly anti-conservative in the simulator (7.5%). Assumption 1 and the exchangeability behind the placebo references are modeling choices that seasonality or product launches can break. T7 depends on neither, which is why we recommend periodic canary campaigns wherever write access exists. Daily e-values are not strictly independent. Finally, Proposition 1 bounds what any of this can achieve against a pipeline that leaks deliberately and sparsely. Code audits, data lineage and attested execution remain necessary complements.

10 Conclusion

Opt-out compliance in black-box graph feature pipelines cannot be proven from outputs, but it can be falsified efficiently. The most effective evidence comes from nodes whose correct output is known in advance: natural canaries and invariant neighborhoods. These carry over from synthetic to real data unchanged. Validity, however, depends on details: typed edges, an agreed opt-out policy, symmetric references, and units that respect how real customers browse. Our real-data null study exposed the last of these, which the simulator could not. Every audit should therefore begin with a null study on real data, and its result should be reported as a calibrated detection bound.

References

Angrist, J. D., and Pischke, J.-S. 2009. *Mostly Harmless Econometrics*. Princeton University Press.

Borisyuk, F.; He, S.; Ouyang, Y.; Ramezani, M.; Du, P.; et al. 2024. LiGNN: Graph neural networks at LinkedIn. In *KDD*.

Biega, A. J., and Finck, M. 2021. Reviving purpose limitation and data minimisation in data-driven systems. *Technology and Regulation* 2021:44–61.

Bourtoule, L.; Chandrasekaran, V.; Choquette-Choo, C. A.; et al. 2021. Machine unlearning. In *IEEE Symposium on Security and Privacy*.

Carlini, N.; Liu, C.; Erlingsson, Ú.; Kos, J.; and Song, D. 2019. The secret sharer: Evaluating and testing unintended memorization in neural networks. In *USENIX Security Symposium*.

Carlini, N.; Chien, S.; Nasr, M.; Song, S.; Terzis, A.; and Tramèr, F. 2022. Membership inference attacks from first principles. In *IEEE Symposium on Security and Privacy*.

Chen, M.; Zhang, Z.; Wang, T.; Backes, M.; Humbert, M.; and Zhang, Y. 2022a. Graph unlearning. In *ACM CCS*.

Chen, C.; Sun, F.; Zhang, M.; and Ding, B. 2022b. Recommendation unlearning. In *The Web Conference (WWW)*.

Chernozhukov, V.; Chetverikov, D.; Demirer, M.; Duflo, E.; Hansen, C.; Newey, W.; and Robins, J. 2018. Double/debiased machine learning for treatment and structural parameters. *The Econometrics Journal* 21(1).

Eisenhofer, T.; Riepel, D.; Chandrasekaran, V.; Ghosh, E.; Ohrimenko, O.; and Papernot, N. 2025. Verifiable and provably secure machine unlearning. In *IEEE SaTML*.

El-Kishky, A.; Markovich, T.; Park, S.; Verma, C.; Kim, B.; Eskander, R.; Malkov, Y.; Portman, F.; Samaniego, S.; Xiao, Y.; and Haghighi, A. 2022.

TwHIN: Embedding the Twitter heterogeneous information network for personalized recommendation. In *KDD*.

Fey, M., and Lenssen, J. E. 2019. Fast graph representation learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

Frasca, F.; Rossi, E.; Eynard, D.; Chamberlain, B.; Bronstein, M.; and Monti, F. 2020. SIGN: Scalable inception graph neural networks. *arXiv:2004.11198*.

Grbovic, M., and Cheng, H. 2018. Real-time personalization using embeddings for search ranking at Airbnb. In *KDD*.

Ginart, A.; Guan, M.; Valiant, G.; and Zou, J. 2019. Making AI forget you: Data deletion in machine learning. In *NeurIPS*.

Halko, N.; Martinsson, P.-G.; and Tropp, J. A. 2011. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. *SIAM Review* 53(2).

Hamilton, W.; Ying, Z.; and Leskovec, J. 2017. Inductive representation learning on large graphs. In *NeurIPS*.

Hermann, J., and Del Balso, M. 2017. Meet Michelangelo: Uber's machine learning platform. Uber Engineering Blog. `https://www.uber.com/blog/michelangelo-machine-learning-platform/`.

He, X.; Jia, J.; Backes, M.; Gong, N. Z.; and Zhang, Y. 2021. Stealing links from graph neural networks. In *USENIX Security Symposium*.

He, X.; Deng, K.; Wang, X.; Li, Y.; Zhang, Y.; and Wang, M. 2020. LightGCN: Simplifying and powering graph convolution network for recommendation. In *SIGIR*.

Huang, Z.; Gong, N. Z.; and Reiter, M. K. 2024. A general framework for data-use auditing of ML models. In *ACM CCS*.

Jagielski, M.; Ullman, J.; and Oprea, A. 2020. Auditing differentially private machine learning: How private is private SGD? In *NeurIPS*.

Koch, M. 2018. Personalized store feed with vector embeddings. DoorDash Engineering Blog.

Liu, Z.; Iqbal, U.; and Saxena, N. 2024. Opted out, yet tracked: Are regulations enough to protect your privacy? *Proceedings on Privacy Enhancing Technologies* 2024(1):280–299.

Meta. 2024. Engineering at Meta blog post on Privacy Aware Infrastructure and purpose limitation with Policy Zones. `https://engineering.fb.com/2024/08/27/security/privacy-aware-infrastructure-purpose-limitation-meta/`.

Nasr, M.; Song, S.; Thakurta, A.; Papernot, N.; and Carlini, N. 2021. Adversary instantiation: Lower bounds for differentially private machine learning. In *IEEE Symposium on Security and Privacy*.

Olatunji, I. E.; Nejdl, W.; and Khosla, M. 2021. Membership inference attack on graph neural networks. In *IEEE TPS-ISA*.

RetailRocket. 2016. Retailrocket recommender system dataset. Kaggle. `https://www.kaggle.com/retailrocket/ecommerce-dataset`.

Ramdas, A.; Grünwald, P.; Vovk, V.; and Shafer, G. 2023. Game-theoretic statistics and safe anytime-valid inference. *Statistical Science* 38(4).

Sarda, A.; Jain, A.; Liu, I.; et al. 2019. Food discovery with Uber Eats: Using graph learning to power recommendations. Uber Engineering Blog. `https://www.uber.com/blog/uber-eats-graph-learning/`.

Schönemann, P. H. 1966. A generalized solution of the orthogonal Procrustes problem. *Psychometrika* 31(1).

Shafer, G.; Shen, A.; Vereshchagin, N.; and Vovk, V. 2011. Test martingales, Bayes factors and p-values. *Statistical Science* 26(1).

Shah, R. D., and Peters, J. 2020. The hardness of conditional independence testing and the generalised covariance measure. *The Annals of Statistics* 48(3).

Shokri, R.; Stronati, M.; Song, C.; and Shmatikov, V. 2017. Membership inference attacks against machine learning models. In *IEEE Symposium on Security and Privacy*.

Sommer, D. M.; Song, L.; Wagh, S.; and Mittal, P. 2022. Athena: Probabilistic verification of machine unlearning. *Proceedings on Privacy Enhancing Technologies* 2022(3):268–290.

Steinke, T.; Nasr, M.; and Jagielski, M. 2023. Privacy auditing with one (1) training run. In *NeurIPS*.

Alibaba Tianchi. 2015. Repeat buyers prediction after sales promotion (IJCAI-15 competition dataset). `https://tianchi.aliyun.com/`.

Alibaba Tianchi. 2018. User behavior data from Taobao for recommendation. `https://tianchi.aliyun.com/dataset/649`.

Thudi, A.; Jia, H.; Shumailov, I.; and Papernot, N. 2022. On the necessity of auditable algorithmic definitions for machine unlearning. In *USENIX Security Symposium*.

Vovk, V., and Wang, R. 2021. E-values: Calibration, combination and applications. *The Annals of Statistics* 49(3).

Wang, J.; Huang, P.; Zhao, H.; Zhang, Z.; Zhao, B.; and Lee, D. L. 2018. Billion-scale commodity embedding for e-commerce recommendation in Alibaba. In *KDD*.

Ying, R.; He, R.; Chen, K.; Eksombatchai, P.; Hamilton, W. L.; and Leskovec, J. 2018. Graph convolutional neural networks for web-scale recommender systems. In *KDD*.

Zhang, M.; Ren, Z.; Wang, Z.; Ren, P.; Chen, Z.; Hu, P.; and Zhang, Y. 2021. Membership inference attacks against recommender systems. In *ACM CCS*.

Zhu, R.; Zhao, K.; Yang, H.; Lin, W.; Zhou, C.; Ai, B.; Li, Y.; and Zhou, J. 2019. AliGraph: A comprehensive graph neural network platform. *PVLDB* 12(12).